

AWS Well-Architected Framework Review for ROCA Event Management Flows: Payment Security, Concurrency, and Operational Resilience

1 Executive Summary

This Batch 2 Well-Architected Review of ROCA's Event Creation and Event Discovery & RSVP flows identified 16 new architectural gaps and confirmed that all 5 Batch 1 carry-forward gaps are amplified by the financial and concurrency demands of payment processing and real-time capacity management. The most critical findings cluster around three themes: payment security, RSVP concurrency safety, and orchestration fragility for event cancellation cascades.

First, ROCA's Stripe integration has significant security exposure. While Stripe tokenization qualifies the platform for SAQ-A (the simplest PCI self-assessment) [1], the architecture lacks webhook signature verification [2], WAF protection on payment endpoints [3], and the script attack protections now mandated by PCI DSS v4.0.1 Requirements 6.4.3 and 11.6.1 [4]. Combined with the Batch 1 DLQ absence, failed Stripe webhook events—including refund confirmations and dispute notifications—can be permanently lost [5].

Second, the RSVP capacity enforcement logic is vulnerable to race conditions that could oversell event slots. The current conditional check is non-atomic under concurrent submissions [6]; remediation requires PostgreSQL atomic updates with row-level locking (`UPDATE ... WHERE capacity > 0 RETURNING capacity`) as the correctness guarantee [7], supplemented by Redis distributed locks as a throughput optimization rather than a safety mechanism [8, 9]. Idempotent state transitions using conditional WHERE clauses are essential to prevent duplicate confirmations across Lambda retries [10].

Third, the RRULE parsing layer built on `rrule.js` carries well-documented DST and timezone bugs—including infinite loops on unknown timezone strings and incorrect time shifts across DST boundaries [47–52]—requiring input validation, iteration caps, and evaluation of the RFC-5545-compliant `rrule-temporal` library [11]. The total remediation effort across 14 prioritized items is approximately 29 weeks, with a net monthly cost increase of roughly \$50–100 after Graviton migration and caching optimizations offset new infrastructure spend. Critical items—webhook verification, atomic capacity enforcement, Step Functions cancellation sagas, and PCI compliance hardening—should be addressed immediately in the next sprint cycle.

2 Introduction and Scope

This report presents Batch 2 of 4 of the AWS Well-Architected Framework Review for the ROCA (Religious Organization Calendar Application) platform, focusing on two core user journeys: the **Event Creation** flow and the **Event Discovery & RSVP** flow. Batch 1 previously assessed the New Member Onboarding flow and identified 15 architectural gaps across all six pillars. This review evaluates the same six pillars — Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, and Sustainability — against the specific demands of event management, payment processing, and concurrent RSVP handling.

The Event Creation flow is a 6-step wizard encompassing basic information entry, schedule and RRULE recurrence configuration, location and resource allocation, ticketing and Stripe payment setup, volunteer role assignment, and review/publish — with conditional branching for recurring events, paid events, and volunteer coordination [6]. The Event Discovery & RSVP flow is a 5-step journey from browse/search through event detail, RSVP/checkout with capacity and waitlist management, confirmation with receipt generation, and a centralized My Events Dashboard [6]. Both flows operate on a serverless architecture comprising Node.js/Express on AWS Lambda, API Gateway (REST), PostgreSQL 15 on RDS Multi-AZ, Redis 7 on ElastiCache, S3/CloudFront, SES/SNS, Stripe payment integration, WebSocket via Socket.io, and JWT/OAuth 2.0 authentication [12].

The event lifecycle follows a DRAFT → PUBLISHED → ACTIVE → COMPLETED → ARCHIVED state machine, with any of the first three states able to transition to CANCELLED, which cascades to REFUND_PENDING → REFUND_COMPLETE for paid events [6]. The RSVP lifecycle follows PENDING → CONFIRMED → CHECKED_IN → ATTENDED, with branches to CANCELLED, WAITLISTED → PROMOTED → CONFIRMED, and WAITLISTED → EXPIRED [6]. These state machines introduce significant complexity around payment security, concurrency control, and cascading operations that were not present in Batch 1's onboarding flow.

3 Batch 1 Gap Cross-Reference and Compounding Impact

Batch 1 identified five critical architectural gaps that carry forward into Batch 2 with amplified impact. The absence of **AWS Step Functions orchestration** is now critically compounded: Event Creation's 6-step wizard with conditional branching, and the cancellation cascade requiring parallel refund processing, notification fan-out, and volunteer cleanup, all demand the saga pattern that Step Functions provides [13]. Without it, ROCA relies on im-

perative Lambda-to-Lambda calls with no built-in compensation logic for partial failures.

The **missing JWT token rotation** identified in Batch 1 becomes a higher-severity issue when payment endpoints are involved. Stripe checkout flows and webhook endpoints handling financial data require short-lived tokens with automatic rotation to minimize the blast radius of token compromise. The **DLQ absence** is equally critical: Stripe webhook events that fail processing are silently lost, with Stripe only retaining undelivered events for 30 days [5]. Without a dead-letter queue capturing failed webhook invocations, payment status updates (completions, refunds, disputes) can be permanently missed.

The **missing WAF and GuardDuty** gap now exposes payment and RSVP endpoints to SQL injection, cross-site scripting, and automated abuse. AWS WAF should be attached to API Gateway REST APIs to protect against these common web exploits [3], and its absence on endpoints handling Stripe tokens and capacity-sensitive RSVP operations represents a significant security and reliability risk. Finally, the **absent EventBridge event bus** prevents clean decoupling between the Event Creation, RSVP, payment, and notification subsystems — Stripe itself supports sending events directly to Amazon EventBridge as an event destination [14], which would eliminate the need for custom webhook endpoint management entirely.

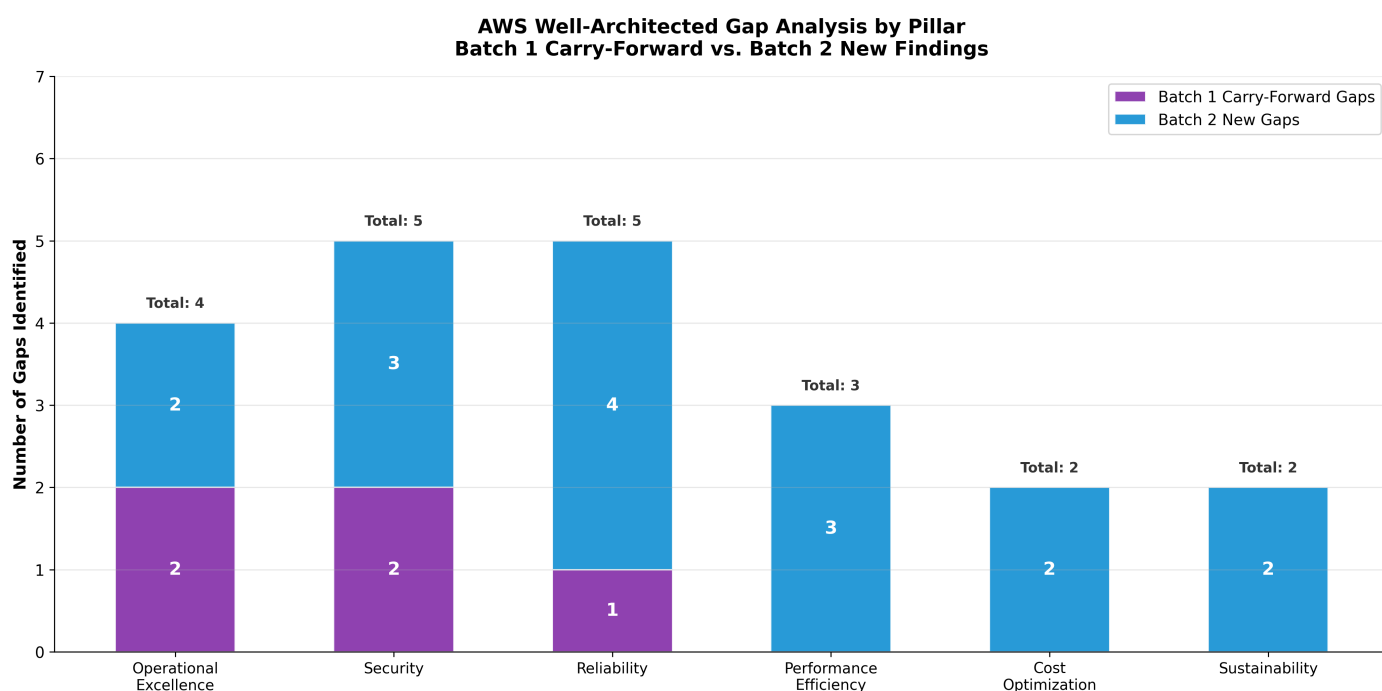


Figure 1: AWS Well-Architected Gap Analysis by Pillar

Chart Explanation: This stacked bar chart shows the distribution of architectural gaps across all six Well-Architected pillars. Batch 1 carry-forward gaps (purple) concentrate in Operational Excellence, Security, and Reliability, while Batch 2 new findings (blue) reveal the heaviest gap concentration in Reliability (4 new gaps including RSVP race conditions and RRULE parsing) and Security (3 new gaps around PCI compliance). Performance Efficiency, Cost Optimization, and Sustainability had no Batch 1 carry-forward gaps but each gained 2–3 new findings in Batch 2.

4 Pillar 1: Operational Excellence

State Machine Observability. Both the event lifecycle (DRAFT → PUBLISHED → ACTIVE → COMPLETED) [6] and RSVP lifecycle (PENDING → CONFIRMED → CHECKED_IN → ATTENDED) [6] lack documented distributed tracing, centralized logging, or operational dashboards. Each state transition should emit structured CloudWatch metrics and EventBridge events to enable real-time monitoring of flow completion rates, transition latencies, and error rates. The volunteer assignment state machine (INVITED → ACCEPTED → CONFIRMED → COMPLETED) [15] adds a third concurrent state machine that must be correlated with event state for operational visibility.

Cancellation Cascade Orchestration. The 4-step cancellation cascade — request, impact analysis, multi-channel notification, and refund processing [6] — is the most operationally complex workflow in Batch 2. The notification timing alone (T+0 in-app/push, T+5min email, T+15min SMS for critical events, T+1hr refund status email) [6] requires precise scheduling that imperative Lambda code cannot reliably deliver. AWS Step Functions with Parallel state should run refund processing, notification fan-out, and volunteer cleanup as concurrent branches with independent error handling [16], while the Map state enables parallel iteration across all affected attendees for batch

refund processing [17]. For high-volume cancellations of recurring event series, Distributed Map mode enables high-concurrency processing with batching [18].

Recurring Event Generation Monitoring. The `event_series` table stores RFC 5545 RRULE strings with exception dates [12], but there is no documented monitoring for RRULE expansion failures, instance generation timeouts, or exception date conflicts. Operational runbooks should define procedures for handling malformed RRULE strings, DST-related generation errors, and cascading updates across event series instances.

5 Pillar 2: Security — Payment Processing and PCI DSS Compliance

PCI DSS v4.0.1 Compliance Posture. ROCA's use of Stripe tokenization means cardholder data never touches ROCA's infrastructure, qualifying the platform for SAQ-A — the simplest PCI self-assessment [1]. The serverless architecture on AWS further reduces compliance burden, with approximately 43% of PCI compliance requirements addressed by the AWS PCI DSS Attestation of Compliance, including time synchronization, file integrity monitoring, patch management, and physical access controls [19]. However, ROCA remains responsible for audit logging (Requirement 10), encryption in transit via TLS 1.2+ (Requirement 4), access controls (Requirement 7), and secure application development (Requirement 6) [19].

Critical Gap: PCI DSS v4.0.1 Script Attack Protection. PCI DSS v4.0.1 now requires merchants to confirm their entire website is secure against malicious script attacks (Requirements 6.4.3 and 11.6.1) [4]. This is a new requirement that ROCA's current architecture does not address. Content Security Policy headers, Subresource Integrity checks, and script monitoring must be implemented on all pages that load Stripe.js elements.

Stripe Webhook Security. Webhook endpoints must verify the Stripe-Signature header using HMAC-SHA256 with the `constructEvent()` function, which includes timestamp-based replay attack prevention [2]. There is no documented implementation of this verification in ROCA's codebase. Combined with the Batch 1 WAF gap, webhook endpoints are exposed to both spoofed events and common web attacks. AWS WAF should provide rate limiting, IP reputation filtering, and payload inspection as a defense-in-depth layer [3, 20].

Payment Data Isolation. The payments table stores `stripe_payment_intent_id` but has no documented encryption at rest beyond RDS default encryption or column-level access controls [12]. While Stripe tokens are not cardholder data, payment intent IDs combined with amounts and user identifiers constitute sensitive financial data requiring access isolation through PostgreSQL row-level security policies and IAM-scoped database credentials rotated via AWS Secrets Manager.

6 Pillar 3: Reliability — Race Conditions, Capacity Enforcement, and RRULE Parsing

RSVP Race Conditions and Capacity Enforcement. The current capacity logic uses a simple conditional check — if `currentAttendees < capacity` then `CONFIRMED`, else if `waitlistEnabled` then `WAITLISTED`, else `REJECTED` [6] — which is vulnerable to race conditions under concurrent RSVP submissions. Two users checking capacity simultaneously could both see available slots and both be confirmed, exceeding the event's capacity limit.

The recommended fix is an atomic PostgreSQL pattern: `UPDATE events SET capacity = capacity - 1 WHERE id = \ $1 AND capacity > 0 RETURNING capacity`, which guarantees no overselling since the WHERE guard and row lock are evaluated atomically [7]. For higher throughput, `SELECT FOR UPDATE SKIP LOCKED` allows concurrent transactions to skip already-locked rows rather than blocking, eliminating deadlocks [21]. A Redis distributed lock (`SET lock:event:{id} {token} NX EX 5`) should serve as a fast gate to throttle concurrent reservation attempts before they hit PostgreSQL, reducing database contention [8]. However, following Martin Kleppmann's analysis, Redis-based locks should be treated as an efficiency optimization rather than a correctness guarantee — the PostgreSQL atomic update must remain the authoritative safety net since Redis locks can expire during GC pauses or clock skew [9].

Idempotent State Transitions. RSVP state transitions should use conditional WHERE clauses: `UPDATE rsvps SET status = 'CONFIRMED' WHERE id = \ $1 AND status = 'PENDING' RETURNING status` ensures the transition fires only once and concurrent retries are safe [10]. For Lambda-level deduplication, AWS Lambda Powertools idempotency utility backed by DynamoDB can deduplicate invocations across retries [22]. PostgreSQL advisory locks (`pg_try_advisory_lock(event_id)`) provide application-level mutual exclusion for coarse-grained operations like waitlist promotion without touching table rows [23].

RRULE Parsing Reliability. The `rrule.js` library, which is the dominant Node.js implementation for RFC 5545 recurrence rules, has well-documented timezone and DST issues. Its internal UTC representation causes recurrence times to shift incorrectly across DST boundaries [24]. Specific bugs include `DTSTART` with `TZID` not following RFC 5545 spec [25], recurrence series changing time-of-day after DST transitions [26], `UNTIL` generated without the re-

quired UTC ‘Z’ suffix when TZID is present [27], the .after() method breaking when TZID is included [28], and unknown timezone strings causing infinite loops [29].

Mitigations should include pre-validation of RRULE strings for FREQ presence, UNTIL/COUNT mutual exclusivity, and TZID against the IANA timezone database before expansion. Iteration must be capped with COUNT or date-range bounds to prevent infinite loops. For a more robust long-term solution, the rrule-temporal library offers the first fully RFC-5545-compliant JS/TS implementation built on the Temporal API with RFC-7529 support [11], while node-ical provides robust RRULE expansion with built-in EXDATE and RECURRENCE-ID support [30].

7 Pillar 4: Performance Efficiency — Search and Checkout Optimization

Event Discovery Search Performance. The Event Discovery flow’s browse/search step [6] requires efficient full-text search across event titles, descriptions, categories, and date ranges. PostgreSQL’s GIN indexes on tsvector columns are the recommended approach, as they are optimized for mapping many values to one row and work well for full-text search and array indexing [31]. PostgreSQL offers a unique convergence of full-text search, vector similarity search, and geospatial search within a single database, enabling rich search experiences without managing multiple specialized systems [32]. Materialized views with REFRESH MATERIALIZED VIEW CONCURRENTLY should pre-compute popular search result sets, and composite indexes combining tsvector with category and location columns will optimize filtered event discovery queries.

Connection Pooling and Lambda Concurrency. Lambda functions connecting directly to RDS risk connection storms under high concurrency. RDS Proxy performs connection pooling and multiplexing that reduces the overhead of opening and closing connections, enabling Lambda functions to reach high concurrency without exhausting database connections [33]. AWS Lambda documentation explicitly recommends RDS Proxy for production workloads [34]. This is especially critical during peak RSVP periods when multiple users simultaneously attempt to register for popular events.

Caching and Edge Optimization. Redis cache-aside with TTL-based expiry (60–300 seconds for search results) balances freshness with cache hit ratio [35]. Event updates should trigger near-real-time cache invalidation via SNS or EventBridge to delete affected cache keys on write. CloudFront cache key policies should include only the minimum necessary values to maximize cache hit ratio [36], with separate cache behaviors for public event listings (longer TTLs, keyed on query parameters) versus authenticated endpoints (short TTLs or cache bypass with Authorization header). CloudFront Functions can normalize cache keys by sorting query parameters and stripping unnecessary headers [37].

Checkout Flow Latency. For the Stripe checkout path, Lambda cold starts can add significant latency to an interactive payment flow. AWS Lambda Provisioned Concurrency pre-initializes execution environments to deliver double-digit millisecond response times, eliminating cold starts for latency-sensitive workloads [38]. Pagination for event browsing should use cursor-based (keyset) pagination rather than OFFSET to avoid inconsistent results when events change status mid-browse, and RDS Proxy’s configurable connection queuing handles burst traffic gracefully [39].

8 Pillar 5: Cost Optimization

Lambda Compute Efficiency. AWS Lambda functions powered by Graviton2 deliver up to 19% better performance at 20% lower cost, achieving up to 34% better price-performance overall [40, 41]. For compute-heavy workloads like recurring event RRULE expansion, switching to Graviton can achieve up to 45% savings [42]. The AWS Lambda Power Tuning tool tests functions across memory configurations from 128 MB to 10 GB and recommends optimal cost/performance settings [43] — this should be applied to all event management Lambda functions, particularly the RRULE expansion and search query handlers.

Provisioned Concurrency Cost Trade-off. Provisioned Concurrency for checkout flows eliminates cold starts but incurs a fixed cost for pre-initialized environments [38]. This should be applied selectively to the Stripe checkout Lambda function and the RSVP capacity enforcement function, where latency directly impacts user experience and payment completion rates. Non-interactive functions (notification dispatch, RRULE batch expansion) should remain on-demand.

Infrastructure Right-Sizing. The Batch 1 recommendation to migrate from REST API to HTTP API on API Gateway (approximately 70% cost reduction) carries forward and applies to all non-WebSocket event management endpoints. ElastiCache Redis should be right-sized based on actual cache utilization metrics, with reserved nodes for predictable baseline capacity and on-demand scaling for event-driven spikes. Batch RRULE expansion during off-peak windows, with results cached in Redis, avoids repeated computation and reduces Lambda invocation costs.

9 Pillar 6: Sustainability

Compute Sustainability. Graviton2-based Lambda functions not only reduce cost but also improve energy efficiency per transaction, aligning with the AWS Well-Architected Sustainability pillar's guidance to minimize environmental impact [40]. Lambda Power Tuning ensures functions are not over-provisioned with memory, reducing wasted compute resources [43]. Recurring event instance generation should be batched during off-peak windows using Step Functions Distributed Map state rather than computed on-demand for each user request, reducing redundant computation across the platform.

Data and Caching Efficiency. Caching expanded RRULE occurrences in Redis eliminates repeated parsing and computation for the same recurrence patterns. Materialized views for popular search queries reduce the number of full-table scans on RDS. CloudFront edge caching for public event listings reduces origin requests and associated compute costs. These patterns collectively reduce the per-request resource footprint of the Event Discovery flow while improving response times.

10 Prioritized Remediation Roadmap

The following table summarizes all identified gaps with severity, estimated effort, approximate monthly cost impact, and the primary Well-Architected pillar addressed.

Priority	Remediation Item	Pillar	Effort	Monthly Cost Impact	Batch 1 Carry-Forward
Critical	Stripe webhook signature verification + WAF deployment [2, 3]	Security	2 weeks	+\$25–50 (WAF)	Yes (WAF)
Critical	RSVP atomic capacity enforcement via PostgreSQL [21, 7]	Reliability	2 weeks	Negligible	No (New)
Critical	Step Functions cancellation saga with parallel refund/notify/cleanup [13, 16]	Operational Excellence	4 weeks	+\$10–30 (Step Functions)	Yes (Step Functions)
Critical	PCI DSS v4.0.1 audit logging and script attack protection [4, 19]	Security	3 weeks	+\$15–25 (CloudWatch)	No (New)
Critical	DLQ for Stripe webhook Lambda processing [5]	Reliability	1 week	+\$2–5 (SQS)	Yes (DLQ)
High	RRULE parsing hardening — validation, iteration caps, library evaluation [24, 29, 11]	Reliability	3 weeks	Negligible	No (New)
High	PostgreSQL GIN indexes + tsvector for event search [31, 32]	Performance	2 weeks	Negligible	No (New)
High	EventBridge event bus for cross-flow decoupling [14]	Operational Excellence	3 weeks	+\$5–15	Yes (EventBridge)
High	RDS Proxy for Lambda connection pooling [33, 34]	Performance	4 weeks	+\$15–30	No (New)
High	Redis distributed lock for RSVP throttling [8, 9]	Reliability	2 weeks	Negligible	No (New)

Priority	Remediation Item	Pillar	Effort	Monthly Cost Impact	Batch 1 Carry-Forward
Medium	CloudFront cache behavior optimization [36, 37]	Performance	2 weeks	-\$5–15 (savings)	No (New)
Medium	Provisioned Concurrency for checkout Lambdas [38]	Cost/Performance	1 week	+\$20–40	No (New)
Medium	Graviton Lambda migration [40, 41]	Sustainability	2 weeks	-\$10–25 (savings)	No (New)
Medium	Lambda Power Tuning for all event functions [43]	Sustainability	1 week	-\$5–15 (savings)	No (New)

Table: Batch 2 Prioritized Remediation Roadmap with Effort and Cost Estimates

**Batch 2 Prioritized Remediation Roadmap
Effort by Severity and Pillar**

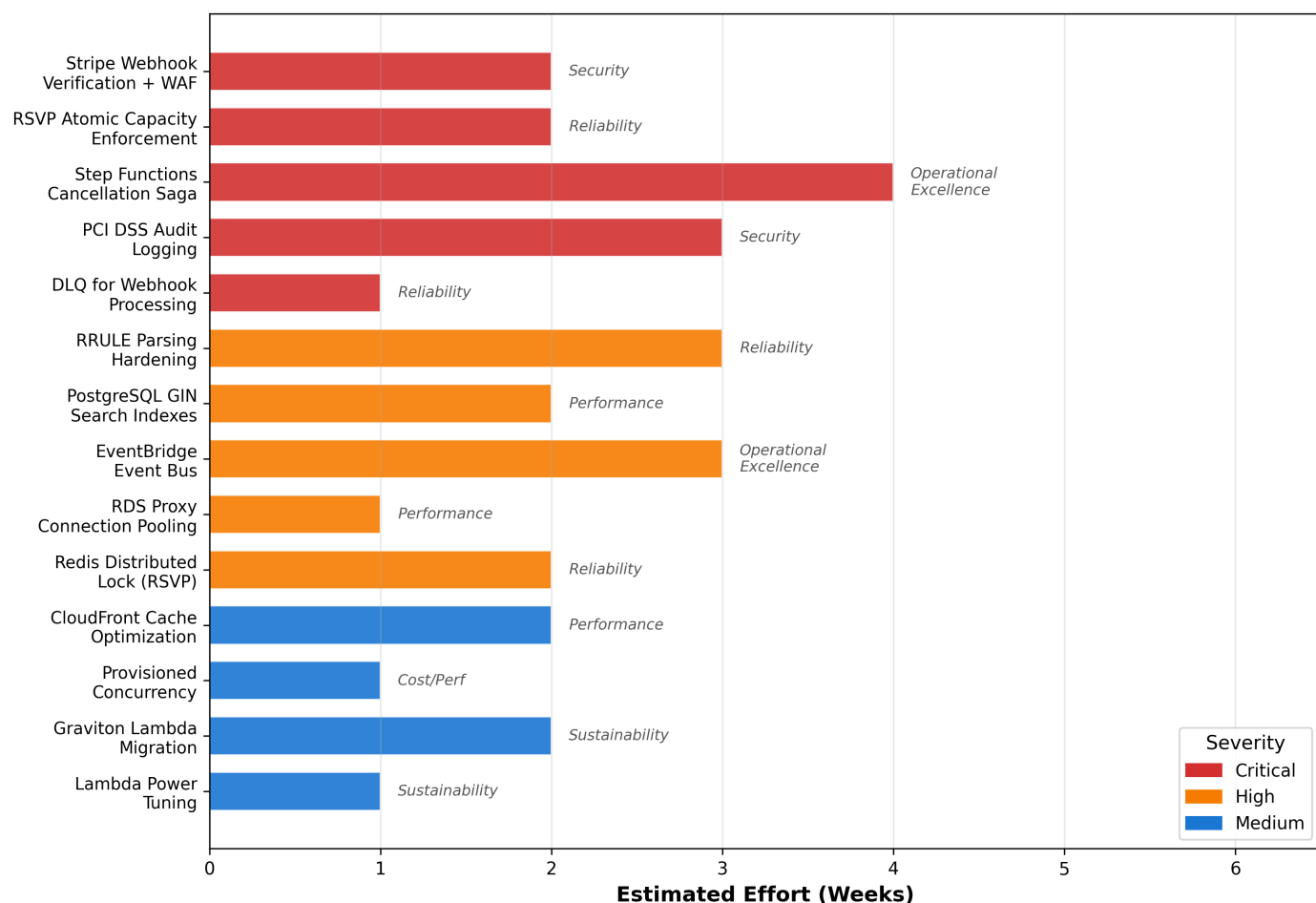


Figure 2: Batch 2 Prioritized Remediation Roadmap

Chart Explanation: This horizontal bar chart visualizes the estimated effort in weeks for each remediation item, color-coded by severity (red for Critical, orange for High, blue for Medium). The five Critical items total approximately 12 weeks of effort and address payment security, capacity enforcement, orchestration, and compliance. High-priority items add 11 weeks focused on RRULE reliability, search performance, and event-driven architecture. Medium-priority items contribute 6 weeks targeting cost optimization and sustainability improvements.

11 Authentication, Authorization, and Token Management

The JWT/OAuth 2.0 implementation requires several enhancements for Batch 2 flows. Payment endpoints must enforce stricter token validation with shorter expiration windows than general API endpoints. The Batch 1 gap around JWT token rotation is amplified here: a compromised token with access to payment or RSVP endpoints could trigger unauthorized refunds or capacity manipulation. Refresh token rotation with one-time-use semantics should be implemented, and Stripe API keys stored in Lambda environment variables must be migrated to AWS Secrets Manager with automatic rotation.

WebSocket authentication for Socket.io — used for real-time event updates and RSVP count broadcasting — requires token validation on connection establishment and periodic re-validation during long-lived connections. Scope-based authorization should differentiate between member-level operations (RSVP, browse) and admin-level operations (event creation, cancellation, refund initiation), with API Gateway authorizers enforcing these scopes before Lambda invocation.

12 Conclusion

Batch 2 reveals 16 new architectural gaps specific to event management, payment processing, and concurrent RSVP handling, in addition to 5 carry-forward gaps from Batch 1 that are now amplified by the financial and concurrency demands of these flows. The most critical findings center on three themes: payment security (PCI DSS v4.0.1 compliance gaps, missing webhook verification, absent WAF), concurrency safety (RSVP race conditions without atomic capacity enforcement, non-idempotent state transitions), and operational orchestration (cancellation cascades without Step Functions saga pattern, RRULE parsing fragility).

The total estimated remediation effort across all 14 items is approximately 29 weeks, with a net monthly infrastructure cost increase of roughly \$50–100 after accounting for optimization savings from Graviton migration, CloudFront caching, and Lambda Power Tuning. Critical items should be addressed in the immediate sprint cycle (Sprints 3–4 per the ROCA task backlog), with High items in the following sprint and Medium items integrated into ongoing optimization work. Batch 3 will assess the Volunteer Scheduling and Scripture/Lectinary Management flows, carrying forward any unresolved gaps from Batches 1 and 2.

13 References

1. <https://stripe.com/resources/more/pqi-tokenization>
2. <https://docs.stripe.com/webhooks/signature?locale=en-GB>
3. <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-control-access-aws-waf.html>
4. <https://www.akamai.com/content/akamai/en/blog/security/2025/feb/pqi-dss-v4-0-1-changes-qualify-saq-a>
5. <https://docs.stripe.com/webhooks/process-undelivered-events>
6. <https://us-east-1.quicksight.aws.amazon.com/sn/spaces/f756a661-a188-4112-bed6-e70567713dbc?documentId=9bf52c4b6-417e-a077-cb09f9214287>
7. <https://www.postgresql.org/docs/8.1/applevel-consistency.html>
8. <https://redis.io/docs/latest/develop/clients/patterns/distributed-locks/>
9. <https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html>
10. <https://docs.aws.amazon.com/wellarchitected/latest/agent-ai-lens/agentrel06-bp04.html>
11. <https://github.com/ggaabe/rrule-temporal>
12. <https://us-east-1.quicksight.aws.amazon.com/sn/spaces/f756a661-a188-4112-bed6-e70567713dbc?documentId=8ed3eb08-5b4f-4464-bf0e-ad8ba2c4e4b5>
13. <https://docs.aws.amazon.com/prescriptive-guidance/latest/patterns/implement-the-serverless-saga-pattern-by-using-aws-step-functions.html>
14. <https://docs.stripe.com/webhooks?dashboard-or-api=api&verify=check-signatures-library>
15. <https://us-east-1.quicksight.aws.amazon.com/sn/spaces/f756a661-a188-4112-bed6-e70567713dbc?documentId=8cca5a93-094c-44d1-864c-91d219d5e621>
16. <https://docs.aws.amazon.com/step-functions/latest/dg/state-parallel.html>
17. <https://docs.aws.amazon.com/step-functions/latest/dg/state-map.html>
18. <https://docs.aws.amazon.com/step-functions/latest/dg/state-map-distributed.html>

19. <https://aws.amazon.com/blogs/security/transforming-transactions-streamlining-pci-compliance-using-aws-serverless-architecture>
20. <https://docs.aws.amazon.com/whitepapers/latest/security-overview-amazon-api-gateway/security-design-principles.html>
21. <https://www.netdata.cloud/academy/update-skip-locked/>
22. <https://docs.aws.amazon.com/powertools/python/stage/utilities/idempotency/>
23. <https://www.postgresql.org/docs/current/explicit-locking.html>
24. <https://github.com/jakubroztocil/rrule/issues/253>
25. <https://github.com/jakubroztocil/rrule/issues/501>
26. <https://github.com/jkbrzt/rrule/issues/550>
27. <https://github.com/jakubroztocil/rrule/issues/440>
28. <https://github.com/jkbrzt/rrule/issues/608>
29. <https://github.com/jakubroztocil/rrule/blob/master/CHANGELOG.md>
30. <https://github.com/jens-maus/node-ical/>
31. <https://docs.aws.amazon.com/dms/latest/oracle-to-aurora-postgresql-migration-playbook/chap-oracle-aurora-pg.tables.indexes.html>
32. <https://aws.amazon.com/blogs/database/building-a-job-search-engine-with-postgresqls-advanced-search-features/>
33. <https://docs.aws.amazon.com/prescriptive-guidance/latest/amazon-rds-proxy>
34. https://docs.aws.amazon.com/en_us/lambda/latest/dg/services-rds.html
35. <https://aws.amazon.com/getting-started/hands-on/boosting-mysql-database-performance-with-amazon-elasticache-for-redis/>
36. <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/understanding-the-cache-key.html>
37. <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/edge-functions-choosing.html>
38. <https://docs.aws.amazon.com/lambda/latest/dg/provisioned-concurrency.html>
39. https://docs.aws.amazon.com/en_us/AmazonRDS/latest/UserGuide/rds-proxy-best-practices.usage-scenarios.html
40. <https://aws.amazon.com/blogs/aws/aws-lambda-functions-powered-by-aws-graviton2-processor-run-your-functions-on-arm-and-get-up-to-34-better-price-performance/>
41. <https://aws.amazon.com/en/lambda/pricing/>
42. <https://docs.aws.amazon.com/prescriptive-guidance/latest/optimize-costs-microsoft-workloads/net-graviton.html>
43. <https://github.com/alexcasalboni/aws-lambda-power-tuning/blob/master/README-SAR.md>